"An Introduction to Prolog"
Part IV

Viviana Mascardi
viviana.mascardi@unige.it

University of Genoa – DIBRIS



Disclaimer

- This presentation may contain material protected by copyright laws.
- In particular, these lessons are based on the material that Peter Schachte made available on the Web years ago, as part of the Logic and Computation Course at the Computer Science Department of the University Melbourne
- We made this material available on the Web only to ensure timely dissemination among our students, and is meant only for students' personal use.
- Any use different from the students' personal use is prohibited.
- V. Mascardi thanks Professor Schachte for the courtesy of sharing his teaching material on Prolog.

Dibris

1. Managing Nondeterminism
   - Fibonacci numbers

2. Lists and Arithmetic
   - List Maximum
   - List Length
   - List Handling

Managing Nondeterminism

A common Prolog programming error is treating multiple clauses as if they formed an if-then-else construct
This is an erroneous definition of absolute_value:

```
absolute_value(X, X) :- X >= 0.
absolute_value(X, Y) :- Y is -X.
```

```
?- absolute_value(-3, V).
V = 3 ;
No
?- absolute_value(42, V).
V = 42 ;
V = -42 ;
No
```

Managing Nondeterminism

The problem is that the second clause promises that

$$|n| = -n$$

for *any* n, regardless of its sign.

The correct definition:

```
absolute_value(X, X) :- X >= 0.
absolute_value(X, Y) :- X < 0, Y is -X.
```

```
?- absolute_value(-3, V).
V = 3 ;
No
?- absolute_value(42, V).
V = 42 ;
No
```

Managing Nondeterminism

Define a Prolog predicate `maximum(X,Y,Z)` such that `Z` is the larger of `X` and `Y`

Managing Nondeterminism

Define a Prolog predicate `maximum(X,Y,Z)` such that `Z` is the larger of `X` and `Y`

```
maximum(X, Y, X) :-        % maximum(x,y) = x if
    X >= Y.                % x >= y
maximum(X, Y, Y) :-        % maximum(x,y) = y if
    X < Y.                 % x < y
```

Fibonacci numbers

Fibonacci numbers: 1, 1, 2, 3, 5, 8, 13, ...

```
fib(0, 1).
fib(1, 1).
fib(N, F) :-
N1 is N - 1,
N2 is N - 2,
fib(N1, F1),
fib(N2, F2),
F is F1 + F2.
```

What if we change the order of goals? What if we do `F is F1 + F2` first? Note: each call makes *two* recursive calls. Are there better ways to compute Fibonacci numbers?

Dibris

Lists and Arithmetic

Summing a list combines arithmetic with list processing:

```
sumlist([], 0).              %sum([]) = 0
sumlist([X|Xs], Sum):-       %sum([X|Xs]) = Sum if
  sumlist(Xs, Sum1),         %sum(Xs) = Sum1 and
  Sum is Sum1 + X.           %Sum = Sum1 + X
```

This can be done more efficiently:

```
sumlist(L, Sum):-                % sum/2 defined in term of sum/3
  sumlist(L, 0, Sum).            % (third arg is the accumulator)
sumlist([], Sum, Sum).             %sum([]) + Sum = Sum
sumlist([X|Xs], Sum0, Sum):-     %sum([X|Xs]) + Sum0 = Sum if
  Sum1 is Sum0 + X,               %Sum1 = Sum0 + X, and
  sumlist(Xs, Sum1, Sum).          %sum(Xs) + Sum1 = Sum
```

See the lecture on efficiency for more on this

Dibris

List Maximum

Compute the maximum element of a list

```
maxlist([X], X).
maxlist([X|Xs], Max) :-
  maxlist(Xs, Max1),
  maximum(Max1, X, Max).
```

Note: no clause for []! Base case is singleton list because `[]` *has* no maximum

List Length

The *length*/2 predicate relates a list to its length

```
?- length([a,b,c], 3).
Yes

?- length([a,b,c], N).
N = 3 ;
No

?- length(L, 3).
L = [_G263, _G266, _G269] ;
No
```

```
?- length(L, N).
L = []
N = 0 ;

L = [_G278]
N = 1

Yes
```

The length predicate is a built-in in many Prolog implementations, but is a library predicate in SICStus Prolog. You should include the line

```
:- use_module(library(lists)).
```

at the beginning of your SICStus Prolog source code.

List Handling

Many other list predicates can be defined using *append*, *length*.
member/2 could be defined just in terms of append:

```
member(X, L) :-          % X is a member of L, if L of form
  append(_, [X|_], L). % any list ++ [X] ++ any list
```

E is the Nth element of list *L* (counting from 0):

```
element(N, L, E) :-     % E is the Nth element of L
  append(F, [E|_], L), % if exists F where L = F ++ [E] ++ any
  length(F, N).         % and the length of F is N
```

List Handling

T is a list taking the first *N* elements of *L*:

```
take(N, L, T) :-      % T is the first N elements of L
  append(T, _, L),    % if L = T ++ any list
  length(T, N).       % and the length of T in N
```

T is what is left after removing the first *N* elements of *L*:

```
drop(N, L, T) :-      % T is the first N elements of L
  append(F, T, L),    % if L = T ++ any list
  length(F, N).       % and the length of T in N
```

L is a list all of whose elements are E:

```
listof([], _).        % [] is a list of anything
listof([E|Es], E) :- % [E|Es] is a list of Es if
  listof(Es, E).      % Es is
```

Dibris

3 All Solutions Predicates

All Solutions Predicates

Prolog has built-in predicates for collecting solutions.

`findall(`T`,`G`,`L`)` — L is a list of all terms T that satisfy goal G in the order solutions are found; variables in G are left uninstantiated; deterministic.

`setof(`T`,`G`,`L`)` — L is a *non-empty* list of all terms T that satisfy goal G; L is sorted with duplicates removed; variables appearing only in G can be instantiated; can be nondeterministic (with different variable instantiations).

`bagof(`T`,`G`,`L`)` — like `setof/3` but without the sorting.

NB: Template term T is left uninstantiated by all of these predicates